



中山大學
SUN YAT-SEN UNIVERSITY

C++语言标准模板库(STL)

中山大学计算机类专业教程



主讲人：郑贵锋



联系方式：

zhenggf@mail.sysu.edu.cn

目录

CONTENTS

01

STL是什么

02

容器

03

迭代器

04

算法

05

其他 (选讲)



STL是什么

- C++标准库的一部分
- STL的称呼是历史原因导致的，目前的标准中已经没有STL字眼
- 在C++20标准中，STL指的是的如下三章所定义的库：
 - 容器库 (Containers library, Chap. 26)
 - 迭代器库 (Iterators library, Chap. 27)
 - 算法库 (Algorithms library, Chap. 28)

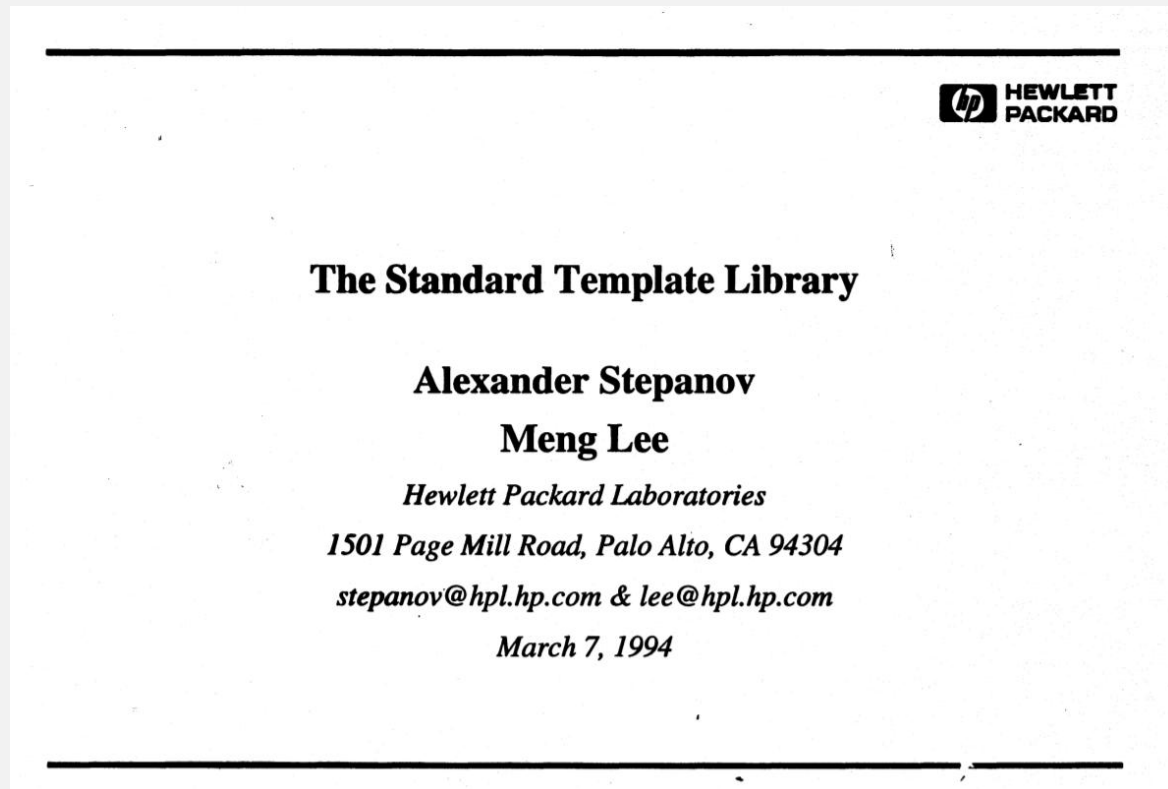
Ref: [C++20标准草稿](#)



STL的历史

1993年，Alex Stepanov开发出STL的原型（Generic C++ Components）。后被C++标准委员会采纳为C++标准的一部分，采纳时的名称叫The **S**tandard **T**emplate **L**ibrary

Ref: [The Standard Template Library](#)

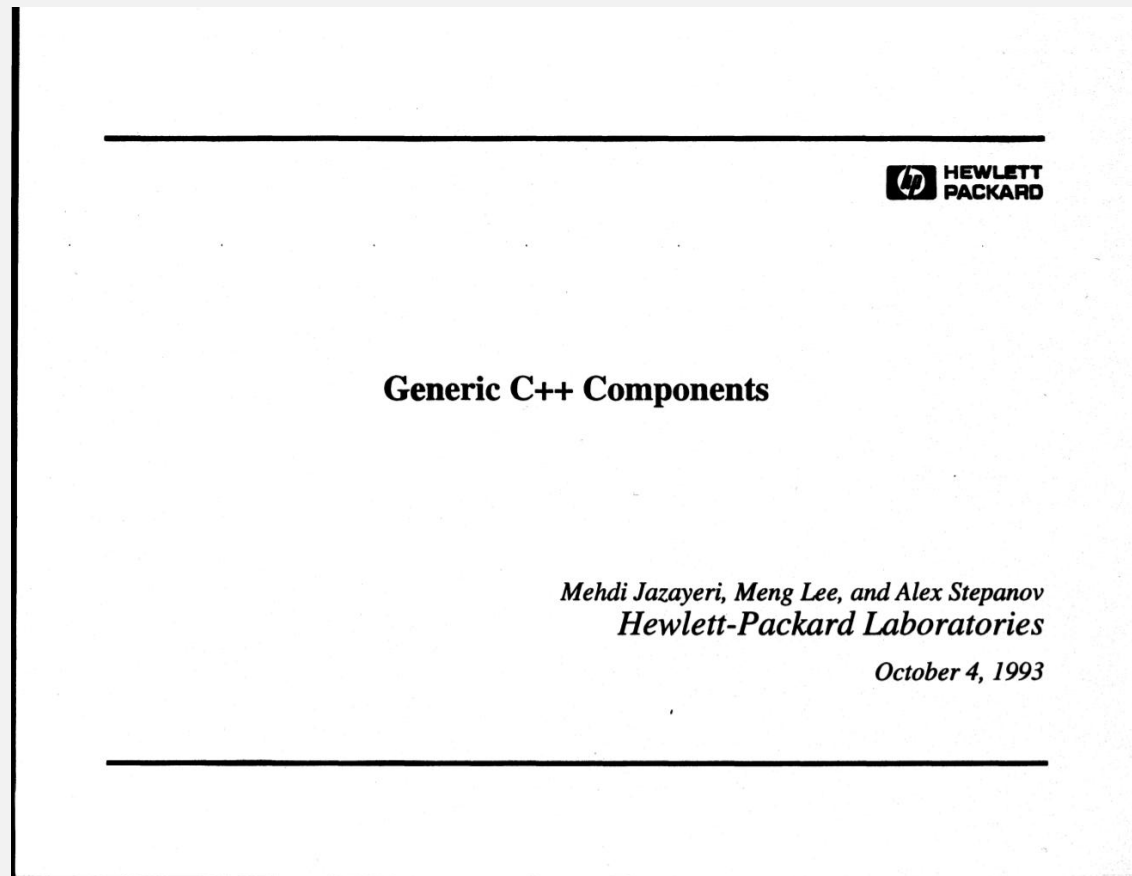




STL的历史

In late 1993, I became aware of a new approach to containers and their use that had been developed by **Alex Stepanov**. The library he was building was called "The STL". Alex then worked at HP Labs but he had earlier worked for a couple of years at Bell Labs, where he had been close to Andrew Koenig and where I had discussed library design and template mechanisms with him. He had inspired me to work harder on generality and efficiency of some of the template mechanisms, but fortunately he failed to convince me to make templates more like Ada generics. Had he succeeded, he wouldn't have been able to design and implement the STL!

Ref: [Evolving a language in and for the real world: C++ 1991-2006](#)





使用容器: exp1.cpp

```
#include <vector>
#include <iostream>
using namespace std;
```

```
int main()
{
```

```
    vector<int> ivec = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
```

```
    ivec.front() = 100;
    cout << "the first element: " << ivec[0] << endl;
```

```
    ivec[1] = 102; // 将第1、第2个元素修改为102、103
    ivec.at(2) = 103;
```

```
    cout << "the second element: " << ivec.at(1) << endl;
    cout << "the third element: " << ivec[2] << endl;
```

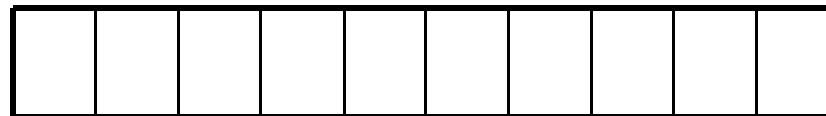
```
    ivec.back() = 999;
```

```
    cout << "the last element: " << ivec[9] << endl;
```

```
}
```

// exp1.cpp

ivec



// ivec包含10个元素，值分别为0~9

// 将第0个元素修改为100
// 输出第0个元素的值

//输出第1、第2个元素的值

//将最后一个元素修改为999

// 输出最后一个元素的值



使用容器: exp1.cpp

ivec

100	102	103	3	4	5	6	7	8	999
-----	-----	-----	---	---	---	---	---	---	-----

the first element: 100
the second element: 102
the third element: 103
the last element: 999

要点解释:

- `vector<T>` 是**动态**的**连续**数组。动态指它的长度是可变的；连续指它的元素可用 `T*` 访问。
- 它实现了两种**元素访问**
 - `operator []`，注意：**不检查下标是否越界**
 - 访问元素函数（返回元素的左值）：`front`，`at(int)`，`back`。其中，**at 检查下标越界**，抛出异常
- 实现了**列表初始化**（直接列表和复制列表）



使用迭代器: exp2.cpp

```
#include <iostream>
#include <vector>
using std::vector;

int main()
{
    vector<int> ivec(10, 2);           // 创建含10个值为2的元素的vector容器
    vector<int>::iterator iter;        // 声明迭代器对象
    vector<int>::reverse_iterator riter; // 声明迭代器对象

    iter = ivec.begin();               // 获取指向第0个元素的迭代器
    *iter += 10;                       // 将第0个元素的值加10

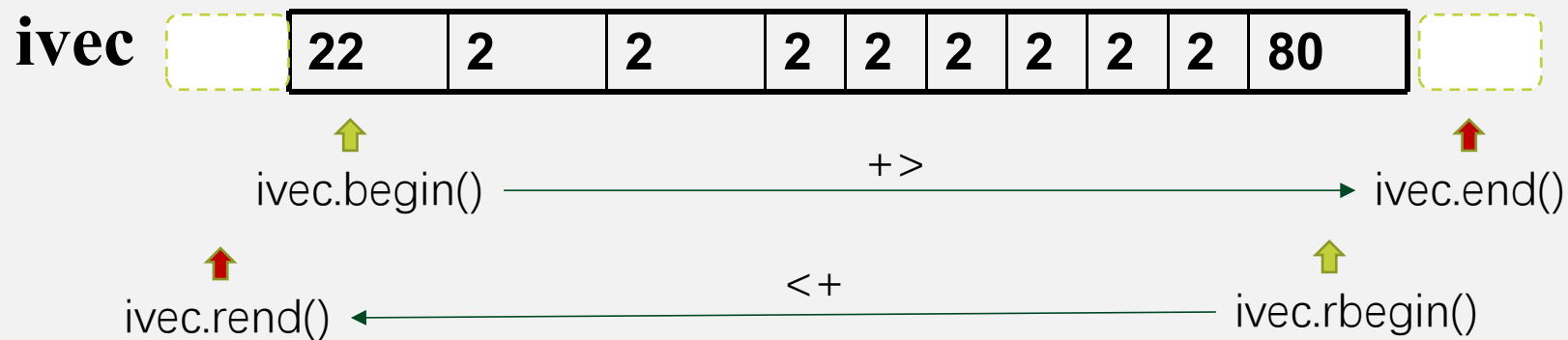
    riter = ivec.rend();               // riter指向第0个元素的前一位置
    *(riter - 1) += 10;               // 将第0个元素的值加10

    iter = ivec.end();                 // iter指向最后一个元素的下一位置
    *(iter - 1) = 100;                // 将最后一个元素的值改为100

    riter = ivec.rbegin();             // riter指向最后一个元素
    *riter -= 20;                      // 将最后一个元素的值减20
}
```




使用迭代器: exp2.cpp



要点解释:

- 迭代器 (*Iterator*) 是用于遍历容器元素的指针对象的包装。
- 单向迭代器实现了以下运算符重载
 - `operator ++`, `==`, `!=`, `*` 等, 例如: `++` 表示取容器中的下一个元素
- 容器会提供一个或多个迭代器实现。例如:
 - `vector` 提供正向和反向的迭代器。
 - 通过成员函数 `begin()`, `end()` 返回正向迭代器对象实例。
 - 通过成员函数 `rbegin()`, `rend()` 返回反向迭代器对象实例



C++11关键字: auto 和 decltype

auto: 占位符类型说明符

对于变量, `auto x = expr`; 从初始化器推导类型

decltype 说明符

`decltype (expr)` 申明从表达式得到的类型

```
vector<int> ivec(10, 2);  
decltype(ivec.begin()) iter;  
  
for (auto it = ivec.begin(); it != ivec.end(); it++ ) {  
    std::cout << *it << ", " ;  
}  
std::cout << std::endl;
```

灵活使用 auto 和 decltype 可以减少程序对模板实参的依赖, 提升程序通用性和可读性



扩展阅读：反向迭代器示例（选讲）

```
template <typename T>                                //template-reverse-it.cpp
class ReverseIterator {
    T* p;
public:
    ReverseIterator(T* x) :p(x) {}
    ReverseIterator(const ReverseIterator& mit) : p(mit.p) {}
    ReverseIterator& operator++() {--p;return *this;}
    ReverseIterator operator++(int) {ReverseIterator tmp(*this);
operator++(); return tmp;}
    bool operator==(const ReverseIterator& rhs) const {return p==rhs.p;}
    bool operator!=(const ReverseIterator& rhs) const {return p!=rhs.p;}
    T& operator*() {return *p;}
};
```

基本概念：容器与迭代器

C++标准：Containers are objects that store other objects. （容器是储存其他对象的对象）

容器分为四种：

- 序列式容器
- 关联式容器
- 无序关联式容器
- 容器适配器

简单来说，每个容器提供了一种适配大部分类型的数据结构

26.3	Sequence containers	<array>
		<deque>
		<forward_list>
		<list>
		<vector>
26.4	Associative containers	<map>
		<set>
26.5	Unordered associative containers	<unordered_map>
		<unordered_set>
26.6	Container adaptors	<queue>
		<stack>

基本概念：容器与迭代器

迭代器：迭代器是每种容器各自定义的一个或多个不同于容器的类，比如 `vector<T>::iterator`，它主要用于访问、修改、增加、删除容器中的元素。

按照功能的不同，C++17之前的迭代器分为: `LegacyInputIterator`（输入迭代器），`LegacyOutputIterator`（输出迭代器），`LegacyForwardIterator`（单向迭代器），`LegacyBidirectionalIterator`（双向迭代器），`LegacyRandomAccessIterator`（随机迭代器）五种，C++17加入了 `LegacyContiguousIterator`（连续迭代器）

Iterator category					Defined operations
LegacyContiguousIterator	LegacyRandomAccessIterator	LegacyBidirectionalIterator	LegacyForwardIterator	LegacyInputIterator	• read • increment (without multiple passes)
					• increment (with multiple passes)
					• decrement
					• random access
					• contiguous storage
Iterators that fall into one of the above categories and also meet the requirements of LegacyOutputIterator are called mutable iterators.					
LegacyOutputIterator					• write • increment (without multiple passes)

目录

CONTENTS

01

STL是什么

02

容器

2.1

顺序容器

2.2

关联容器

2.3

适配器

03

迭代器

04

算法

05

其他 (选讲)



顺序容器

顺序容器

顺序容器实现能按顺序访问的数据结构。

<code>array (C++11)</code>	静态的连续数组 (类模板)
<code>vector</code>	动态的连续数组 (类模板)
<code>deque</code>	双端队列 (类模板)
<code>forward_list (C++11 起)</code>	单链表 (类模板)
<code>list</code>	双链表 (类模板)

其中：array 是固定长度数组。
其他都是动态数量元素的容器

std::array

在标头 `<array>` 定义

```
template<
    class T,                (C++11 起)
    std::size_t N
> struct array;
```

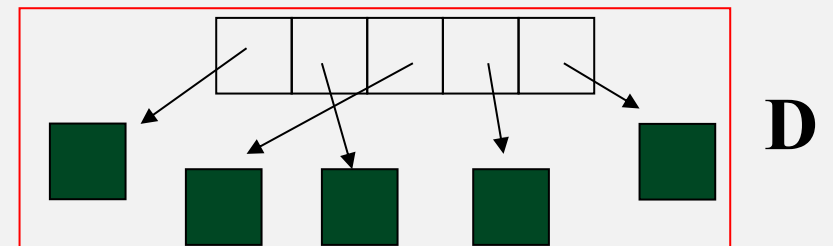
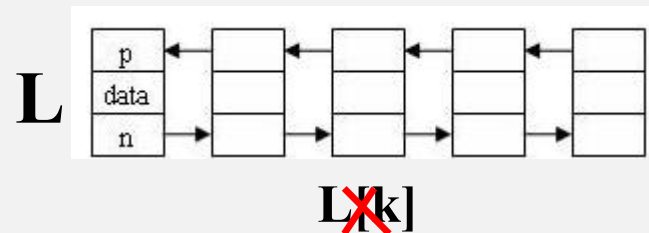
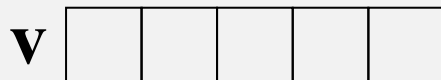
std::array 是封装固定大小数组的容器。

例如：std::array<int, 3> a2 = {1, 2, 3};



序列式容器类型

template	Description (I&D=insertion&deletion)	header
Vector (动态的连续数组)	1. Implemented by array. 2. Fast I&D at the tail. 3. Random access supported. $V[k]$ ✓	<code><vector></code>
List (双链表)	1. Implemented by double linked list. 2. Fast I&D at arbitrary position. 3. Random access NOT supported. $L[k]$ ✗	<code><list></code>
Deque (双端队列)	1. Implemented by pointer array. 2. Fast I&D at both ends. 3. Random access supported. $D[k]$ ✓	<code><deque></code>





序列式容器的构造函数

函数使用形式	说 明
<code>C<T> c;</code>	创建空容器。
<code>C<T> c(cx);</code>	创建容器c作为cx的副本，c和cx必须是同类型且元素类型也相同的容器。
<code>C<T> c(b, e);</code>	创建c，并用迭代器b和e所标示范围内的元素对c进行初始化（c中存放b和e范围内元素的副本）。 注：[b,e)半开区间
<code>C<T> c(n, t);</code>	创建c，并在其中存放n个值为t的元素，t必须是T类型的值，或者可以转换为T类型的值。
<code>C<T> c(n);</code>	创建c，并在其中存放n个元素。每个元素都是T类型的值初始化元素。
<code>C<T> c({...});</code> <code>C<T> c = {...};</code>	创建c，用初始化列表中的元素作为c的元素。这是C++11加入的新方法，最为便利。



容器的构造函数：实例

- 分配指定数目的元素，并对这些元素进行值初始化：

```
vector<int> ivec1(10);
```

// ivec1包含10个0值元素

- 分配指定数目的元素，并将这些元素初始化为指定值：

```
vector<int> ivec2(10, 1);
```

// ivec2包含10个值为1的元素

- 将vector对象初始化为一段元素的副本：

```
int ia[10] = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };
```

```
vector<int> ivec3(ia, ia + 10);
```

// ivec3包含10个元素，值分别为0~9

- 将一个vector对象初始化为另一个vector对象的副本：

```
vector<int> ivec4(ivec3);
```

// ivec4包含值为0~9的元素（与ivec3相同）

- 使用初始化列表： `vector<int> ivec5 = { 1, 2, 3, 4 };` // ivec5包含1~4的元素



访问序列式容器中的元素

使用形式	形式参数	返回值	备 注
<code>c.back()</code>	无	容器c中最后一个元素的引用	若容器为空，则该操作的行为没有定义
<code>c.front()</code>	无	容器c中第0个元素的引用	若容器为空，则该操作的行为没有定义
<code>c[index]</code>	index为元素的下标（元素在容器中序号）	返回下标为index的元素的引用	list容器不提供该操作。若下标越界（即小于0或大于元素数-1），则该操作的行为没有定义
<code>c.at(index)</code>	index为元素的下标	返回下标为index的元素的引用	list容器不提供该操作。进行越界检查

访问序列式容器中的元素：迭代器

使用形式	返回值	备 注
<code>c.begin()</code> <code>c.cbegin()</code>	迭代器。指向容器c中第0个元素	带c的方法，如 <code>c.cbegin()</code> 返回的是常迭代器，这样的迭代器是只读迭代器，只能用于读取元素。 (C++11引入) 其他操作都有两个版本：一个是const成员，一个是非const成员。若容器c为const对象，则这些操作返回的迭代器的类型为带const_前缀的类型（这样的迭代器是只读迭代器，只能用于读取元素，不能用于修改元素）；否则，返回读写迭代器
<code>c.end()</code> <code>c.cend()</code>	迭代器。指向容器c中最后一个元素的下一位置	
<code>c.rbegin()</code> <code>c.crbegin()</code>	逆向迭代器。指向容器c中最后一个元素	
<code>c.rend</code> <code>c.crend()</code>	逆向迭代器。指向容器c中第0个元素的前一位置	



在序列式容器中插入元素

使用形式	形式参数	返回值	操作效果
<code>c.insert(iter, t)</code>	<code>iter</code> 为常迭代器, <code>t</code> 为元素值	指向新插入元素的迭代器	在 <code>iter</code> 所指元素之前插入值为 <code>t</code> 的元素
<code>c.insert(iter, n, t)</code>	<code>iter</code> 为常迭代器, <code>t</code> 为元素值	无	在 <code>iter</code> 所指元素之前插入 <code>n</code> 个值为 <code>t</code> 的元素
<code>c.insert(iter, b, e)</code>	<code>iter</code> 为常迭代器、 <code>b</code> 和 <code>e</code> 均为迭代器	无	在 <code>iter</code> 所指元素之前插入 <code>b</code> 和 <code>e</code> 所指范围内的元素（不包括 <code>e</code> 所指向的元素）
<code>c.push_back(t)</code>	<code>t</code> 为元素值	无	在 <code>c</code> 的尾端增加值为 <code>t</code> 的元素
<code>c.push_front(t)</code>	<code>t</code> 为元素值	无	在 <code>c</code> 的头端增加值为 <code>t</code> 的元素



在序列式容器中插入元素 (1) : addElementsDemo.cpp

```
#include <iostream>
#include <string>
#include <vector>
#include <deque>
using std::deque, std::vector, std::string, std::cout, std::endl;

int main()
{
    vector<int> ivec;                // 创建空的vector容器, 用于存放int型对象
    deque<string> sdeq;              // 创建空的deque容器, 用于存放string型对象
    int iarr[] = {100, 100, 100};

    // 在vector容器中增加元素: 在尾端增加10个元素: 值为1~10
    for (int i = 1; i < 11; i++)
    {
        ivec.push_back(i);
    }
```

ivec

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----



在序列式容器中插入元素 (2) : addElementsDemo.cpp

```
// 在vector容器头端再增加一个元素，值为20  
ivec.insert(ivec.begin(), 20);
```

ivec

20	1	2	3	4	5	6	7	8	9	10
----	---	---	---	---	---	---	---	---	---	----

```
// 在vector容器的第四个元素后再增加两个元素，值均为30  
ivec.insert(ivec.begin() + 4, 2, 30);
```

20	1	2	3	30	30	4	5	6	7	8	9	10
----	---	---	---	----	----	---	---	---	---	---	---	----

```
//将数组iarr中的元素增加到vector容器尾端。注意：被插入的元素不包括  
//第三个参数所指向的元素因此，要插入iarr中的所有元素，第三个参数应  
//该为iarr加3
```

```
ivec.insert(ivec.end(), iarr, iarr + 3);
```

```
// 在deque容器中增加元素
```

```
sdeq.push_back("is");
```

```
sdeq.push_front("this");
```

```
sdeq.insert(sdeq.end(), "an");
```

```
sdeq.insert(sdeq.end(), "example");
```

20	1	2	3	30	30	4	5	6	7	8	9	10	100	100	100
----	---	---	---	----	----	---	---	---	---	---	---	----	-----	-----	-----



在序列式容器中插入元素 (3) : addElementsDemo.cpp

```
// 输出vector容器中的元素
cout << "vector:" << endl;
for (vector<int>::iterator it = ivec.begin(); it != ivec.end(); it++)
    cout << *it << ' ';
cout << endl;

// 输出deque容器中的元素
cout << "double-ended queue:" << endl;
for (deque<string>::iterator it = sdeq.begin(); it != sdeq.end(); it++)
    cout << *it << ' ';

return 0;
}
```

练习：打开cpp中print函数模板，并使用该函数反向输出 ivec 和 sdeq 中的元素



在序列式容器中删除元素

使用形式	形式参数	返回值	操作效果	备 注
<code>c.clear()</code>	无	无	删除容器中的所有元素	
<code>c.erase(iter)</code>	<code>iter</code> 为迭代器	常迭代器。指向被删除元素的下一元素	删除 <code>iter</code> 所指向的元素	若 <code>iter</code> 等于 <code>c.end()</code> ，则该操作的行为没有定义
<code>c.erase(b, e)</code>	<code>b</code> 和 <code>e</code> 为迭代器	常迭代器。指向被删除元素段的下一元素	删除 <code>b</code> 和 <code>e</code> 所指范围内的所有元素（不包括 <code>e</code> 所指向的元素）	
<code>c.pop_back()</code>	无	无	删除容器中最后一个元素	若容器为空，则该操作的行为没有定义
<code>c.pop_front(t)</code>	无	无	删除容器中的第一个元素	若容器为空，则该操作的行为没有定义。 。 <code>vector</code> 容器不提供该操作



在序列式容器中删除元素(1): elementDeleteDemo.cpp

```
int main()
{
    deque<int> ideq = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

```
    // 输出删除操作之前deque容器中的所有元素
    cout << "before delete:" << endl;
    print(ideq.begin(), ideq.end());
```

```
    // 删除容器中的第一个及最后一个元素
    ideq.pop_front();
    ideq.pop_back();
```

```
    auto iter = ideq.begin();
    ideq.erase(ideq.erase(iter + 1));
```

2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---

```
    // iter指向ideq中现存的第0个元素
    // 删除ideq中现存的第1、第2个元素
    // 输出删除操作之后list容器中的所有元素
```

2	5	6	7	8	9
---	---	---	---	---	---



在序列式容器中删除元素(2): elementDeleteDemo.cpp

```
// 删除容器中现存的前三个元素
ideq.erase(ideq.begin(), ideq.begin() + 3);

// 输出删除操作之后list容器中的所有元素
cout << "three elements at front are deleted:" << endl;
print(ideq.begin(), ideq.end());

// 删除剩余的所有元素
ideq.clear();
cout << "after clear:" << endl;
if (ideq.empty()) // 容器为空
    cout << "no element in double-ended queue" << endl;

return 0;
}
```

7	8	9
---	---	---

序列式容器的比较操作

比较操作	比较结果
<code>==</code>	若两个容器中的元素个数相同且对应位置上的每个元素都相等，则比较结果为 <code>true</code> ，否则为 <code>false</code>
<code>!=</code>	结果与 <code>==</code> 操作相反
<code><</code> , <code><=</code> , <code>></code> , <code>>=</code>	若一个容器中的所有元素与另一容器中开头一段元素对应相等，则较短的容器小于另一容器；否则，两个容器中第一对不相等元素的比较结果就是容器的比较结果



序列式容器比较(1): containerCompare.cpp

```
int main()
{
    int iarr[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    list<int> ilist1(iarr, iarr+10);
    list<int> ilist2(iarr, iarr+5);
    list<int> ilist3(ilist2);
    list<int> ilist4(ilist2);

    ilist4.push_back(12);
    ilist4.push_back(7);

    //print all
    vector<string> name = {"list1","list2","list3","list4"};
    vector<list<int>> vec_list = {ilist1,ilist2,ilist3,ilist4};
    for (int i = 0; i < 4; i++) {
        cout << name[i] << ": ";
        print(vec_list[i].begin(),vec_list[i].end());
    }
}
```

list1: 1 2 3 4 5 6 7 8 9 10
list2: 1 2 3 4 5
list3: 1 2 3 4 5
list4: 1 2 3 4 5 12 7



序列式容器比较(2): containerCompare.cpp

```
//containerCompare
cout << "ilist2 == ilist3 : ";
if (ilist2 == ilist3)
    cout << "true" << endl;
else
    cout << "false" << endl;

cout << "ilist1 < ilist2 : ";
if (ilist1 < ilist2)
    cout << "true" << endl;
else
    cout << "false" << endl;

//。 。 。

}
```

list1: 1 2 3 4 5 6 7 8 9 10
list2: 1 2 3 4 5
list3: 1 2 3 4 5
list4: 1 2 3 4 5 12 7



序列式容器的容量操作

使用形式	形式参数	返回值	操作效果
<code>c.empty()</code>	无	若容器为空，则返回 <code>true</code> ；否则返回 <code>false</code>	
<code>c.size()</code>	无	返回容器中目前所存放的元素的数目，类型为 <code>C::size_type</code>	
<code>c.max_size()</code>	无	返回容器中可存放元素的最大数目，类型为 <code>C::size_type</code>	
<code>c.resize(n)</code>	<code>n</code> 为元素数目	无	将容器的大小调整为可存放 <code>n</code> 个元素。若 <code>n < c.size()</code> ，则删除多余元素；否则，在尾端增加相应数目的新元素，新元素均采用值初始化
<code>c.resize(n, t)</code>	<code>n</code> 为元素数目， <code>t</code> 为元素值	无	新增元素取值为 <code>t</code> ，其余效果同 <code>c.resize(n)</code>



序列式容器的赋值和交换

使用形式	形式参数	操作效果	备 注
<code>c1 = c2</code>	c2为容器	首先删除c1中的所有元素，然后将c2中的元素复制给c1。	c1和c2必须是同类型容器且其元素类型也必须相同。
<code>c.assign(b, e)</code>	b和e为一对迭代器	首先删除c中的所有元素，然后将迭代器b和e所指范围内的元素复制到c中。	b和e不能指向c中的元素。 b和e所指范围内元素的类型不必与c的元素类型相同，只需类型兼容即可。
<code>c.assign(n, t)</code>	n为元素数目，t为元素值	首先删除c中的所有元素，然后在c中存放n个值为t的元素。	
<code>c1.swap(c2)</code>	c2为容器	交换c1和c2的所有元素。实际上是c1与c2交换名称。	c1和c2必须是同类型容器且其元素类型也必须相同。

目录

CONTENTS

01

STL是什么

02

容器

2.1

顺序容器

2.2

关联容器

2.3

适配器

03

迭代器

04

算法

05

其他 (选讲)



关联式容器

常见字典数据

- 学生名册
- 电话簿
- 词典 (维基百科)
- 书目
- ...

Key: No.	Mapped: Name
05110000	Zhang San
05110001	Li Si
05110002	Wang Wu
05110003	Lin Liu

pair

Set

基础术语

- Map/Dict 映射/字典
- Key 关键字
- Value 映射值
- Pair/Item 对/项
- Set 集合
 - 元素无顺序
 - 元素不重复



关联式容器

类 名	说 明	所在头文件
map	通过键进行元素存取的相关数组	<map>
multimap	支持重复键的相关数组	<map>
set	任意元素的集合	<set>
multiset	可以重复的集合	<set>
unordered_map	无序的map，类似于哈希表	<unordered_map>
unordered_set	无序的set，类似于哈希表	<unordered_set>



std::pair

std::map的某些函数使用了<utility>中的类std::pair，pair<T1, T2>代表一个由类型T1和类型T2组成的有序对。

可以直接用构造函数 std::pair<T1, T2> (v1, v2) 进行构造，也可以用make_pair进行构造：

```
auto p = make_pair(v1, v2);
```

通过**p.first**访问第一个元素，通过**p.second**访问第二个元素



map的构造函数

函数使用形式	说 明
<code>map<K, T> m;</code>	创建空的map容器m，其键类型为K，值类型为T
<code>map<K, T> m(mx);</code>	创建map容器m作为mx的副本。m和mx的键类型和值类型都必须相同
<code>map<K, T> m(b, e);</code>	创建map容器m，并用迭代器b和e所标示范围内的元素对m进行初始化（m中存放b和e范围内元素的副本）
<code>map<K, T> m = { {...}, {...} }</code>	通过初始化列表创建map，最为简便的方法。（C++11加入）



向map中插入元素

用`map::insert`插入，一共有9种重载。

```
std::map<string, int> m = { { "hello", 1 } };  
m.insert({ "h", 10 });  
m.insert(std::pair{"Kageyama", 180.6});
```

返回值为一个`std::pair`，第一个元素为一个指向被插入元素的迭代器，第二个元素指示插入是否成功。

最简便的方法是直接用`operator[]`来插入元素：

```
m["me"] = 10
```

事实上，由于插入和修改这两个核心操作都可以用这种写法，我们推荐这种写法以保证代码的洁净。



在map中查找元素

使用形式	形式参数	返回值
<code>m.find(k)</code>	k为要查找的键	若容器m中存在与k对应的元素，则返回指向该元素的迭代器；否则，返回指向m中最后一个元素的下一位置的迭代器(即 <code>m.end()</code>)。
<code>m.count(k)</code>	k为要查找的键	k在容器m中的出现次数。



在map中删除元素

使用形式	形式参数	返回值	操作效果	备 注
<code>m.erase(k)</code>	<code>k</code> 为要删除元素的键	被删除元素的个数，其类型为map容器中定义的size_type	若容器m中存在键为k的元素，则删除该元素	若返回值为0，表示要删除的元素不存在
<code>m.erase(iter)</code>	<code>iter</code> 为指向要删除元素的迭代器	无	删除 <code>iter</code> 所指向的元素	若 <code>iter</code> 等于 <code>c.end()</code> ，则该操作的行为没有定义
<code>m.erase(b, e)</code>	<code>b</code> 和 <code>e</code> 为迭代器，表示要删除元素的范围	无	删除 <code>b</code> 和 <code>e</code> 所指向范围内的所有元素（不包括 <code>e</code> 所指向的元素）	要么 <code>b</code> 和 <code>e</code> 相等（此时删除范围为空，不删除任何元素），要么 <code>b</code> 所指向的元素出现在 <code>e</code> 所指向的元素之前



map的实例演示：电话簿

```
*****
1--insert          2--delete a item
3--search          4--delete some items
5--modify          6--display
0--quit
*****
Enter your choice:1
Enter the name you want to insert: ma
Enter the phone number(s) : 12-3122-1

*****
1--insert          2--delete a item
3--search          4--delete some items
5--modify          6--display
0--quit
*****
Enter your choice:3
Enter the name you want to search: ma
The phone number of ma is 12-3122-1
```

multimap

- 不支持下标操作。
- insert操作每调用一次都会增加新的元素
(multimap容器中, 键相同的元素相邻存放)。
- 以键值为参数的erase操作删除该键所关联的所有元素, 并返回被删除元素的数目。
- count操作返回指定键的出现次数。
- find操作返回的迭代器指向与被查找键关联的第一个元素。
- 结合使用count和find操作依次访问multimap容器中与特定键关联的所有元素。

ChenQi	84111111
LiSi	84111112
LiSi	84111111
LiSi	84111234
LiSi	84110100
Wangwu	84111100
ZhangSan	84111123



multimap

使用形式	说 明	备 注
<code>m.lower_bound(k)</code>	该操作返回一个迭代器，指向容器m中第一个键 $\geq k$ 的元素	若键k在容器中不存在，则这两个操作所返回的迭代器相同，都指向k应该插入的位置
<code>m.upper_bound(k)</code>	该操作返回一个迭代器，指向容器m中第一个键 $> k$ 的元素	
<code>m.equal_range(k)</code>	该操作返回包含一对迭代器的pair对象，其first成员等价于 <code>m.lower_bound(k)</code> ，其second成员则等价于 <code>m.upper_bound(k)</code>	



set

map支持的操作set基本上都支持，但有区别。如下：

- 1) 不支持下标操作。
- 2) 没有定义mapped_type类型
- 3) set容器定义的value_type类型不是pair类型，而是与key_type相同，指的都set中元素的类型。

```
std::set<int> s = { 1, 2, 3, 4 };  
  
auto iter1 = s.find(3);  
auto iter2 = s.insert(3); // 返回值为指向之前的3的迭代器，没有实际插入发生  
auto b = iter1 == iter2; // b 为 true
```